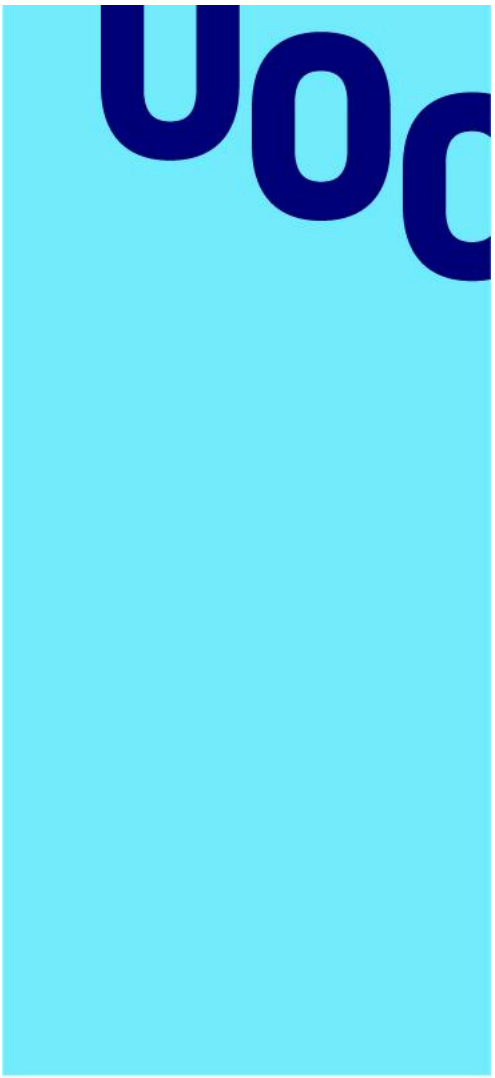
End-to-End Automated Cloud Data Pipeline

Eloi Sanchez Ambros



Universitat Oberta de Catalunya

Index

Final Work Card

Title of the work:	End-to-End Automated Cloud Data Pipeline
Name of the author:	Eloi Sanchez Ambros
Name of the tutor:	Francesc Julbe López
Name of the SRP:	Susana Acedo Nadal
Date of delivery:	28th December, 2025
Studies or Program:	Master in Data Science
Area of the Final Work:	Data Engineering
Language of the work:	English
Keywords:	pipeline, automation, orchestration, cloud, python, azure

Abstract

In the last decades, data has become an important asset to exploit by both public and private organizations. Data is being generated by virtually every device and application at an ever increasing rate, but its value cannot be harnessed without the proper infrastructure to store it and process it. In this project, a Cloud Pipeline is designed and implemented to take raw data from ingestion to a curated form. Moreover, the pipeline is hosted in the cloud and fully automated to produce fresh data on a daily basis. The pipeline is logically split into three main steps. Ingestion takes new data from the OpenWeatherMap API and stores it in a raw format. Staging parses the data and stores it in an analytics-oriented manner as parquet files. Finally, a Transformation step processes the data through a Medallion Architecture, from the Bronze layer into a ready-to-use form in the Gold layer for further downstream applications. Two data products are obtained from this pipeline, a reporting table with aggregated KPIs that are automatically updated daily, and a Machine Learning study for rain prediction evaluating two classifier models (Random Forest and Gradient Boosting). The result is a fully Automated Cloud Data Pipeline that successfully fetches, ingests and processes data which is subsequently fed into downstream applications. The financial details of the project, from the beginning to the end, are also reported and analysed, and alternatives and further steps that would further improve the pipeline are discussed.

Contents

Chapter 1

Introduction

1.1. Context and Justification of Work

In recent decades, the sentence “Data is the new Oil”, coined by mathematician Clive Humby back in 2006[1], has become generally accepted. This belief has only increased with the appearance of data products in general markets, being generative Artificial Intelligence (AI) the latest driver of this trend in recent years.

This has not only modified market dynamics to the point that technological companies have become the dominating businesses[2], but it has also shaped how public agencies gather information to make the best decisions for the people[3]. Figure 1 shows how much each sector weighs in the Standard and Poor 500s index, which includes the 500 biggest companies in the United States’ economy. As of October 9th 2025, the Information Technology (IT) sector, which includes the data sector, is leading with a weight of almost three times the second leading sector.

Information Technology (69)	34.25%
Financials (75)	13.54%
Communication Services (24)	10.77%
Consumer Discretionary (50)	10.74%
Health Care (60)	8.83%
Industrials (78)	8.04%
Consumer Staples (37)	5.32%
Energy (22)	2.77%
Utilities (31)	2.29%
Real Estate (31)	1.80%
Materials (26)	1.71%

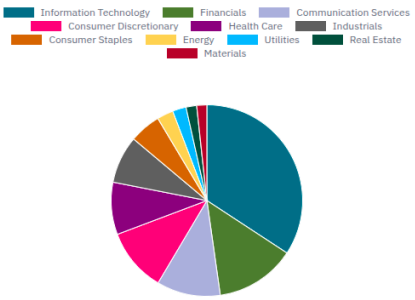


Figure 1. Standard and Poor’s 500 market index grouped by sector as of 09-10-2025.

[1] Arthur, “Tech Giants May Be Huge, but Nothing Matches Big Data.”
[2] US500, “S&P 500 Companies By Sector.”
[3] “DATOS Y BARCELONA | Ciencia e Innovación | Ajuntament de Barcelona.”

However, just as with Oil, data is not of much use in its raw state. It must be obtained, processed and transformed into derived products in order to be useful. The specifics on how to perform these steps has been in constant evolution since the data beginnings of the Era of Data.

The need of creating a whole data warehouse in order to support decisions in business was first seen at the beginning of the 70s. Since then multiple different methodologies, tools and architectures have been developed and adopted, mainly centered around a business data warehouse[4]. Besides having multiple standardized architectures, i.e. Inmon[5], Kimball[6] or Data Vault[7], organizations now have to choose whether to have their data warehouses hosted locally or on the cloud, and there's an immense amount of tools that can be used to do so.

There are multiple steps that must be planned and implemented before an organization can start generating data driven products, which are often less recognized than the final products itself. However, without a data warehouse architecture no dashboards, predictive analytics or Machine Learning (ML) models can be implemented[8].

Nevertheless, and even though all this is well known by both public and private organizations, recent reports show that companies are still not utilizing their data at its full potential[9] and are experiencing significant losses[10]. Therefore, it is important to have professionals capable of designing and implementing data architectures and pipelines that extract value out of raw data adapting to business needs.

The focus of this project is the creation of an end-to-end data reporting pipeline that allows the construction of a user-facing usable data product starting from the ingestion of raw data. Instead of just focusing on a data product (dashboard, ML model…) emphasis is put on the creation of an automated data pipeline that makes possible the construction of such a product starting from zero. For this purpose, the pipeline will be conceptually differentiated into three different parts: ingestion, transformation and orchestration.

[4] Rodero et al., "The Audit of the Data Warehouse Framework."
[5] Inmon, *Building the Data Warehouse*.
[6] Kimball et al., *The Data Warehouse Lifecycle Toolkit*.
[7] Linstedt, "Data Vault Series 1 – Data Vault Overview."
[8] Perspectives, "Why Data Matters."
[9] Salesforce, 73% of Business Leaders Believe Data Reduces Uncertainty and Drives Better Decisions — So Why Aren't They Using It?
[10] Moore, "How To Create A Business Case For Data Quality Improvement."

1.2. Work Objectives

In the following thesis, a data pipeline will be designed and implemented in order to ingest, process and transform data in a scheduled and automated manner. This kind of pipeline, as discussed in the previous section and further elaborated in the following ones, is in great demand in both private and public organizations, since they are the foundation for all further data applications.

For this end, weather data obtained from OpenWeatherMap[11] is going to be ingested, processed and transformed. Moreover, the data coming out of the pipeline is going to be used to calculate a number of Key Performance Indicators (KPIs) that could be used in a reporting dashboard and/or application, and to train two predictive models in a Machine Learning study.

More specifically, the project will work around fulfilling the following objectives:

- **Data Ingestion:** Two different data sources will be ingested into the data pipeline: General weather data[12] and Air pollution data[13].
- **Data Cleaning and Transformation:** The raw data sources will be cleaned and transformed into tabular and readily usable data by other data products, such as dashboarding and reporting tools or machine learning models.
- **Data Products:** The clean data will be used to produce two data products. First a reporting table that aggregates the raw data in order to calculate a set of KPIs is going to be manufactured. Furthermore, data coming out of the pipeline is going to be used to perform a Machine Learning study.
- **Pipeline Cloud Orchestration:** The three steps above will be “orchestrated”, i.e. the Ingestion and Transformation processes will be automated in the Cloud so that new data is introduced on an automatic and periodic basis. This will ensure that the Data Products are up-to-date and ready to be consumed by users without human intervention.

[11] “OpenWeatherMap.”
[12] “Historical Weather API - OpenWeatherMap.”
[13] “Air Pollution API - OpenWeatherMap.”

1.3. Impact on Sustainability, Ethics, Society and Diversity

This work has a strong focus on cloud technologies, computational power for data mobility and transformation and continuous and automated execution of the data pipeline. Therefore, sustainability issues have arisen and been addressed, and its impact on the environment minimized. Cloud technologies already had a carbon footprint higher than the Airline industry as a whole, with each datacenter consuming the electricity equivalent of 50000 homes[14].

Users of these technologies and, more importantly, large-scale companies and organizations must ensure that usage of cloud technologies is minimal and well-justified. It is because of this that the cloud architecture on this project has been designed to be as minimal as needed, and technologies used have been chosen taking into account the scale of the project at hand.

Moreover, in a society where generative AI is constantly being pushed both to mainstream users and to code developers, recent studies show great caveats that must be taken into account when adopting these tools.

Firstly, there are rising concerns on the effects that generative AI may be having in the environment due to their vast consumption of resources, both during the training phase and inference[15,16]. Secondly, extensive usage of generative AI for essay writing has been found to be correlated with a long-term negative effect on the cognitive abilities of users[17]. Finally, contrary to what economic and ML experts predicted, using AI for developing code has been found to actually reduce the productivity of experienced developers[18]. For all these reasons, the usage of generative AI has been completely avoided for the development of this project in all its phases (research, architecture design, code development and writing).

In understanding and planning to minimize the environmental effects this project may have, it will explicitly adhere to the following Sustainable Development Goals (SDGs), on the topic of **Sustainability**, defined in the A/RES/70/1 Resolution of the United Nations[19]:

- 11 - Sustainable cities and communities
- 12 - Responsible consumption and production
- 14 - Life below water
- 15 - Life on land

[14] Monserrate, “The Cloud Is Material.”
[15] Strubell et al., “Energy and Policy Considerations for Deep Learning in NLP.”
[16] Bashir et al., “The Climate and Sustainability Implications of Generative AI.”
[17] Kosmyna et al., “Your Brain on ChatGPT.”
[18] “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity.”
[19] *A/RES/70/1 Transforming Our World: The 2030 Agenda for Sustainable Development*.

As for the SDGs regarding **Ethical behaviour and Social Responsibility**, and **Diversity and Human Rights**, this project has no direct relation to any of them. Nevertheless, it lies within a context of a complex society affected by issues that fall under the SDGs, including, but not limited to, poverty, hunger, lack of peace, gender issues and material inequalities. All efforts have been made in order to avoid exacerbating any issue that negatively affects society, whether it is addressed by the SDGs or not.

1.4. Focus and Methodology

Given the wide array of possibilities available in order to create a data pipeline that fulfills all the objectives of the project, a significant part of the development time was expected to be spent in developing multiple proof of concepts to test out different architectures and tools, and to analyse their viability in fulfilling our needs. The tasks required in order to complete a successful project are shown in Figure 2, and have been organized in the following way:

- **Pipeline minimal viable product**
  - Research and development of multiple aspects of the data pipeline, including ingestion, transformation and orchestration.
  - Creation of tests and prototypes with different tools in order to study their viability if necessary.
  - Obtention of a minimal cloud pipeline capable of ingesting and transforming data at a schedule.

- **Pipeline finalization**
  - Finalise data pipeline, ensuring data is clean and usable.
  - Make sure the data pipeline scales when using production amounts of data and is maintainable in case additions and/or modifications must be performed during the creation of the data products.

- **Data products**
  - Create the two representative data products defined in the objectives.

- **Manuscript and presentation**
  - Finalize writing the manuscript.
  - Prepare presentation and defense of the project.

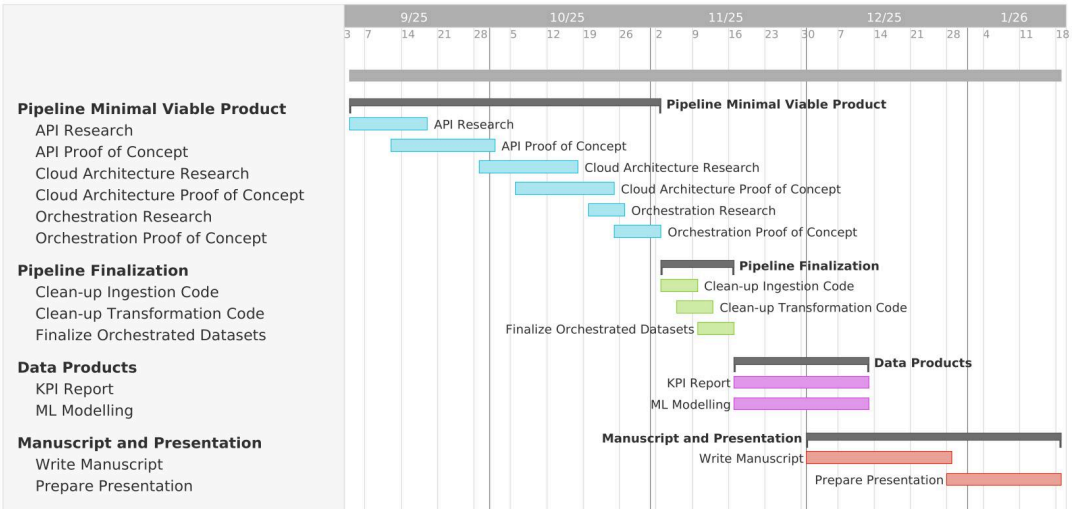


Figure 2. Gantt representation of the expected work timeline[20].

[20] "TeamGantt."

1.5. Work Schedule

In the previous section, the main tasks have been dissected into smaller subtasks. Moreover, in Figure 2, a planned schedule for each of the tasks is laid out, and is further detailed next:

- **From September to end of October – Pipeline minimal viable product**
  - API ingestion, Cloud Architecture and Orchestration tasks have been differentiated into research and development phases. In all cases an iterative process of investigation, developing proof of concepts and analysing their viability should be followed.

- **From November to mid-November – Pipeline finalization**
  - Once the technologies and architectures are minimally decided and working, effort will be put in ensuring the final data is ready by the moment the data products need to be created.
  - When ingestion and transformation are decided to be ready for production, scheduling should be implemented using the architecture and tools investigated in the previous step.

- **From mid-November to mid-December – Data products**
  - Manufacture a reporting table with calculated KPIs.
  - Create a Machine Learning study using the data obtained from the pipeline.
  - Start formalization of manuscript.

- **From mid-December to mid-January – Manuscript and presentation**
  - Notes and sections of the Manuscript will be drafted as the project develops so that the last weeks of the project are used only for finalizing the text.
  - Presentation will be prepared once the majority, or all, of the manuscript is finalized.

Given the technical and unforeseen difficulties that may arise during the development of complex IT architectures such as the one in this project, this schedule is taken as an idealised guide that may not be strictly followed. Agility and adaptability are preferred over deeply rigid long-term planning.

1.6. Products Obtained

- **Data pipeline:** The data pipeline, defined as the joint processes of Ingestion, Transformation and Orchestration.
    - **Ingestion** is the process of introducing data from a source into the pipeline. A PULL based ingestion process has been developed where data is fetched from the OpenWeatherMap API at a schedule.
    - **Transformation** is the phase where raw data, already ingested in the pipeline, is transformed into usable, somewhat structured data that can be directly accessed by data products. In this project the medallion structure has been used and, moreover, a separate Staging process has been added in between Ingestion and Transformation (Section 3.3).
    - **Orchestration** refers to the execution of all the steps executed by the pipeline at a schedule. This process is what ensures that data is updated and readily available to the data products whenever is needed. The pipeline in this project has been scheduled to be executed at 08:30 Europe/Amsterdam every workday.
- **Ingestion CLI and general APIs**: The source code used for each of the steps has been designed in a way that can be easily used and configured by an end user with limited technical knowledge. Three distinct interfaces have been designed and are described in Section 3. Moreover, a Command Line Interface (CLI) to facilitate one time executions of Ingestion processes has been developed.
- **Reporting Table**: A reporting table including aggregated data with observable KPIs that other tools can readily use (dashboarding, reporting apps…) has been manufactured from the ingested raw data.
- **Machine Learning Study**: Using the clean and transformed data obtained from the data pipeline, a Machine Learning study will be created as proof that the data coming from the pipeline is indeed usable and of good quality.

1.7. Thesis Structure

In the current sections an introduction has been provided contextualizing the inspirations, objectives and importance of this project, as well as how it fits in the current political context within the context of the Sustainability Goals directed by the United Nations. Moreover, a set of main objectives have been laid down and a general plan on how the work has been structured has been presented.

In Section 2 a more detailed discussion on the state of the art of reporting pipelines is presented, as well as an in-depth explanation of the different possibilities that have been investigated during the development of the project.

Section 3 provides detailed information of the developed pipeline, the tools used and the implementation that has been used in each of the steps. Moreover, details on the derived data products will be shown, displaying how useful an automated data pipeline can be, and displaying some examples on the wide array of possibilities that arise from being able to obtain clean and usable data from raw sources.

Finally, Section 4 evaluates whether the main objectives of the thesis have been achieved and investigates which parts of the architecture could be modified in order to improve the reliability or capabilities of the pipeline. Future efforts are explored, analysing the current state of the pipeline, the data-derived products and possibilities of scaling the infrastructure in order to include further sources.

Chapter 2

State of the Art

As described in the previous sections, there are multiple ways to design and develop a reporting data pipeline. An endless list of tools and theoretical designs exist, and good care has to be taken in order to fit the solution to the specific needs of the project. Therefore, it is important to understand the current state of technology and best design and architectural practices. In this section a methodical analysis of the possible decisions that were available for each main part of the project are laid down.

2.1. Hosting Platforms

Originally, companies hosted their IT infrastructure in their own data centers. These could be dramatically different in size and specifications depending on the company's needs and technological investment, and could range from small servers in a closet in the building to giant server racks that occupied rooms or even full buildings with special cooling. In fact, some companies still host their own on-premise data centers.

However, hosting one's own data center can require a significant capital expenditure in order to buy and build the needed hardware, followed by a constant operational expenditure in order to power, maintain, upgrade and secure the hardware and software. Furthermore, it comes with the need of hiring professionals dedicated to maintaining these servers. Often, this may be too much of a cost for an organization with limited resources[21].

During the last decades, cloud environment usage has been rapidly rising due to multiple benefits against on-premise data centers, i.e., managed security, pay-per-use, immediate scalability…[22] Therefore, even though a project of this scale could be easily handled by an ordinary, on-premise machine, a cloud infrastructure has been implemented.

This is in order to take into account one of the big concerns of IT infrastructure: scaling up. Creating a whole data infrastructure with local tools for the only reason that it is enough for the current organization needs can prove to be a mistake in the long term. Typically, organizations grow, and with growth comes more data, more needs and bigger resource demand. One of the biggest advantages of cloud over on-premise data centers are almost infinite storage and compute elasticity[23], which are pay-per-use in many of the cloud services and allow scaling up without having to acquire and maintain any extra hardware.

There are multiple cloud providers that are either generally purposed (Amazon Web Services, Google Cloud Platform, Microsoft Azure) or more specifically tailored to some specific data needs (Snowflake, Vertica).

General Cloud platforms work in terms of micro-services. In essence, they provide hundreds of different products with different degrees of configurability and functionalities in order to allow organizations to choose from a wide array of solutions that could fit their needs the best. Moreover, these microservices are designed in order to facilitate, to certain extent, integrations with each other. Therefore, Cloud architects must plan ahead which microservices are going to be used and how they are going to be integrated with each other.

[21] Font, "A Complete Newbies Primer on the Data Center Ecosystem."
[22] Deloitte Insights, "Why Organizations Are Moving to the Cloud."
[23] "What Is Elastic Computing - Definition | Microsoft Azure."

2.2. Ingestion

During the Ingestion process, data is taken from a source and introduced into the organization's data platform for further treatment. It can take many different forms depending on three factors: the origin of the data, how often data is ingested and how the ingestion process is triggered.

2.2.1. Sources

Sources can be of a wide array of forms. Some examples are provided here.

- **Transaction systems:** A very typical form of source are transaction logging systems, e.g. sales made by a shop that are logged into a transactional database. In this case, ingestion into the data warehouse will require transferring the new rows logged in the transactional system into an analytical database.
- **Web Application Programming Interfaces (APIs):** Although APIs generally refer to an Interface layer that allows programmatic communication to a certain system, here the focus is put on data retrieval Web APIs. Typically, these APIs provide an interface to communicate with some kind of data storage system, which provides data to the client via different public endpoints.
    - **Software as a Service**: Organizations may pay for products that they use in order to operate their companies (payments systems, human resource management, project management softwares…). The data generated by this organization can be obtained and loaded to the warehouse by accessing the public endpoints provided by these products, given that proper authorization is provided during access.
    - **Data Providers**: There are companies or organizations that provide data as a service, which could be free or paid. The data used in this project, for instance, comes from the OpenWeatherMap API, which provides different tiers with different levels of access to multiple APIs[24].
- **Internet of Things (IoT) devices:** IoT devices consist of the network of devices that typically include one or more sensors, compute power and transmitting capabilities. Applications using these devices are widely different but, in general, they tend to provide big amounts of realtime, or nearly realtime, data[25].

[24] "Weather API - OpenWeatherMap."
[25] Shafiq et al., "The Rise of 'Internet of Things.'"

2.2.2. Ingestion Trigger

There are two main ways in which to classify ingestion depending on its frequency. On one hand, ingestion can be triggered on a schedule determined by the data ingestion program. Alternatively, the ingestion process can always be open to receiving new data, which is sent by the source[26].

- **PULL:** On PULL based ingestion, data is "pulled" by the ingestion process from the source and ingested into the database. This could be set to be manually triggered when needed or, more typically, be automated so that the ingestion takes place at a certain schedule (hourly, daily, weekly…)
- **PUSH:** On PUSH based ingestion, data is "pushed" by the data source into the database, which is constantly ready to receive the data. In this case, the source could be sending the data to the destination as soon as it gets there, becoming, in this case, a real-time ingestion.

2.2.3. Ingestion Frequency

Finally, depending on how often data is ingested, it can be considered classified into batch ingestion or real-time ingestion[27].

- **Batch:** In this case a batch of new data is ingested every time the ingestion process is triggered. Generally, PULL ingestions are all of this kind, since they will ingest the new batches of data whenever they are triggered. Moreover, PUSH ingestions where the source sends data to the target database at given intervals or after a certain amount of new data has been introduced can also be considered of this type.
- **(Near) Real-time:** When PUSH ingestions are set up so that the source sends data to the target database as soon as it is generated they are considered to be real-time, or near real-time, ingestions. There are very few cases where the benefits of having real-time data outweigh the costs of having these systems running, e.g., fraud detection, stock trading…[28]

[26] Reis and Housley, *Fundamentals of Data Engineering*.
[27] Reis and Housley, *Fundamentals of Data Engineering*.
[28] "What Is Real-Time Data?"

2.3. Data Modelling

Once data is introduced into the database, there are multiple standardized ways to organize it. Here we discuss a few of the most common data modelling techniques from the past decades[29].

- **Inmon[30]:** Designed in 1989 by Bill Inmon, this modelling technique stores data in a highly normalized (3NF) data warehouse that acts as a single source of truth. Analyses, when required, are generated downstream of this data warehouse via department-specific data marts, where denormalization may occur to speed up client-sided queries.
- **Kimball[31]:** In the early 1990s, Ralph Kimball developed this data modelling technique that can accept denormalization. The Kimball model intends to serve departments from the data warehouse itself, organizing data in fact tables, which contain factual, quantitative and event-related data, that are enriched by dimension tables, which provide context for the fact tables. This way of organizing data is called Star Schema.
- **Data Vault 2.0[32]:** Also appearing in the 1990s, created by Dan Linstedt, this modelling strategy separates the structural aspects of data from its attributes. It includes three types of tables:
    - **Hubs**: Insert only tables that contain all unique business keys loaded.
    - **Links**: Track relationships of business keys between hubs at the lowest possible grain.
    - **Satellites**: Include descriptive attributes that give meaning and context to hubs. They can connect either to hubs or links.
- **Medallion[33]:** The Medallion architecture, which is the one chosen for this project, is typically used as a way to organize data in a lakehouse. In this architecture, data is organized in three layers with the goal of progressively improving the data as it goes through the layers of the architecture:
    - **Bronze**: Raw data “lands” in this layer and is not typically modified, but load datetime, process ID, or other information that could help in tracking or auditing the data can be added.
    - **Silver**: The data from the Bronze layer is transformed in a way that the result provides an “Enterprise view” of its entities, concepts and transactions. The data

[29] Reis and Housley, *Fundamentals of Data Engineering*.
[30] Inmon, *Building the Data Warehouse*.
[31] Kimball et al., *The Data Warehouse Lifecycle Toolkit*.
[32] Linstedt, “Data Vault Series 1 – Data Vault Overview.”
[33] Databricks, “What Is a Medallion Architecture?”

coming out of this layer is meant to provide the building blocks for reporting, advanced analytics and ML.

- **Gold**: Using the business building blocks created in the Silver layer, data is further processed to obtain consumption-ready and project-specific tables or marts so that access to the very specific information it provides is as quick as possible.

2.4. Data Applications

Data Applications are the end products of a data pipeline. The creation of these products are the reason why all the previous steps are performed and are typically the product that is finally used or presented to clients or users.

The nature of these applications can take any shape or form, and here only a few examples are presented:

- **Reports and Dashboards:** These are distilled graphical representations of data. They are designed in a way that answers one or more questions in a quick and comprehensive way. As an example, a dashboard could show a sales summary and trends of the past week or month. The leader of a company would look at this dashboard on a daily basis in order to make business decisions.
- **Outlier notifications:** Many systems require human intervention when there is a malfunction, or alerts could be needed if certain conditions occur. Applications can be built on top of processed log data with algorithms that send notifications to the responsible people. Examples of this could be an application built on top of a financial system that notify the bank in case that fraud is detected or an application that notifies the meteorological agency if anomalies are detected in measured weather data.
- **Artificial Intelligence**: With the rise of Large Language Models, companies have found a way to save on man power by creating support chatbots accessible through their webpages. These chatbots can be built from AI models that require data structured in a certain way in order to be processed, and their answers can only be as good as the data that is provided to them during the training process.

Chapter 3

Results

3.1. Overall Architecture

Three generalistic cloud providers were considered for this project: Amazon Web Services (AWS), Google Cloud Platform (GCP) and Microsoft Azure. These three platforms include essentially the same services, with minor differences in form, and are not specifically tailored to any purpose in particular. Instead, they offer a wide range of microservices that can be used and integrated with each other to fit the needs of almost any project.

In the end, Microsoft Azure was the platform selected for this project due to their offering of 100$ worth of credits to spend during a year for a student-linked account. This, in combination with the selection of products with free usage (with limitations) offered during the first year after the student account linking, made it a valid choice for the scope of this project[34].

Overall, the project is conceptually divided into three main blocks: Ingestion, Staging and Transformation. Moreover, these three blocks must be automated to run at a schedule so that end-users and applications relying on the pipeline always have fresh data. In Figure 3, a visual representation of the pipeline design is shown.

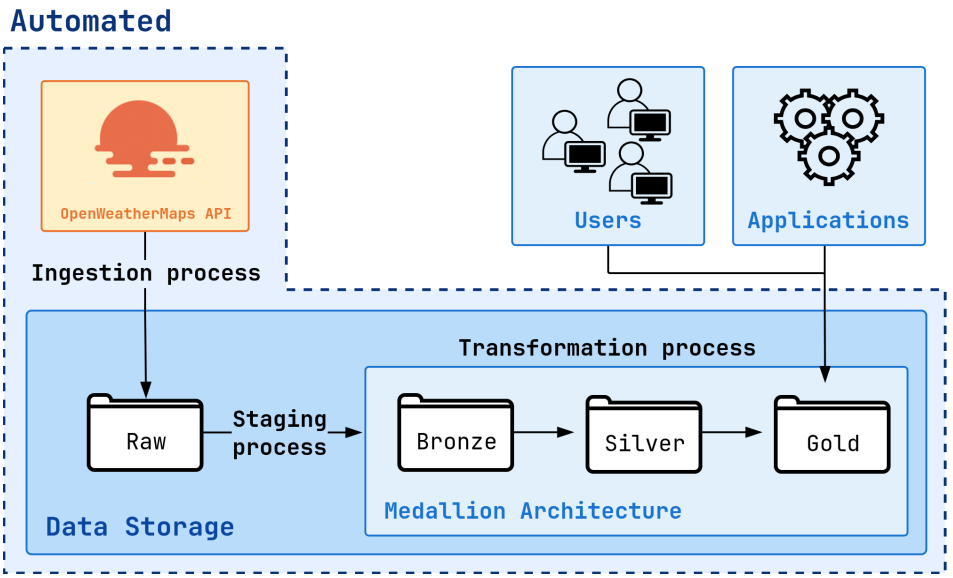


Figure 3. Diagram of the overall design of the automated data pipeline.

[34] "Azure for Students | Microsoft Azure."

3.1.1. Cloud Architecture

Multiple services have been used for this project, which can be divided into three main categories.

Storage

Azure provides multiple storage solutions, but Azure Data Lake Storage (ADLS) was the one utilized for this project. It is built on top of Blob Storage, the main storage solution provided by Azure, but includes multiple extra capabilities that are useful for Big Data Analytics[35].

The main advantage of ADLS over Blob Storage is that it includes a “Hierarchical directory structure” optimized for high-performance data access. This allows optimized and performant access of data in well organized and partitioned containers, as we will have in the landing stage of the ingestion step.

Even though we will be far from working with what is considered Big Data, and for our current purposes Blob Storage could have been used instead without a big impact in performance, the cost of using ADLS is low enough that it is worth choosing it over Blob Storage in order to not worry about scaling issues in the future.

Computation

There are various computing options available for choosing in Azure. These vary wildly depending on their focus, from very generalistic Virtual Machines to Big Data SaaS offerings.

Initially, Databricks was planned to be the main computational provider for data transformation. However, after the initial Proof of Concept, when Databricks was added to the account at the beginning of October, it was found that it cost more than 1$ per day, even when idling. Therefore, other options were considered.

The main purpose of the computation would be to execute a few script-like applications at most once a day. Moreover, these executions were expected to be short, especially at the beginning, given that data would be limited during the development process.

Due to these facts, generalistic virtual machines that would be available during the whole day were discarded in search for something that could be triggered at specific moments.

[35] “Azure Data Lake Storage Introduction - Azure Storage.”

Azure also offers other database management systems, both transaction- and analytics-oriented. However, the configuration of such database platforms can be complex and maintenance can be costly, as shown in Section 3.7.

Instead, Azure Functions, the Serverless Compute option from Azure, appeared as a fitting solution. These processes are paid by the minute of use, of course taking into account the size of the machines that will be available. They also provide native support for Python, which is already the main language of this project. Once configured and deployed, functions can be triggered via API calls to a provided URL, given that the call is properly authenticated.

Orchestration

Finally, having decided on solutions for Storage and Computation, the only remaining requirement is to have an automated execution of the pipeline at a schedule, so that we always have fresh data ready for use.

Solutions like database management systems or constantly running virtual machines would provide some native orchestration tools, like task or cron jobs. Moreover, other tools could be installed in virtual machines, such as Airflow or Dagster. However, since we relied on the limited Serverless Compute Pools as a computational solution, orchestration must be configured in another way.

The main orchestrating solution provided by Azure is Azure Data Factory (ADF), which integrates well with other Azure solutions like Functions. In essence, ADF allows creating sequential flows that trigger procedures from other Azure services.

In the current scenario, the flow, called “Pipeline” in ADF, is quite simple, with sequential calls to the three functions that handle Ingestion, Staging and Transformation.

Final Cloud Architecture

The final Cloud Architecture can be seen in Figure 4.

As can be seen, there are three functions that handle Ingestion, Staging and Transformation. Details of their implementation are provided in the following section.

ADF was scheduled to execute the three functions in order on a daily basis, excluding weekends, simulating an organization that needs fresh data on a daily basis.

The Ingestion function will fetch data from the OpenWeatherMap API and store it in the raw container of ADLS. Once this is done, sequential calls to the Staging and Transformation functions will be triggered, which will transform the data in different ways and populate the other containers with clean and ready to use data.

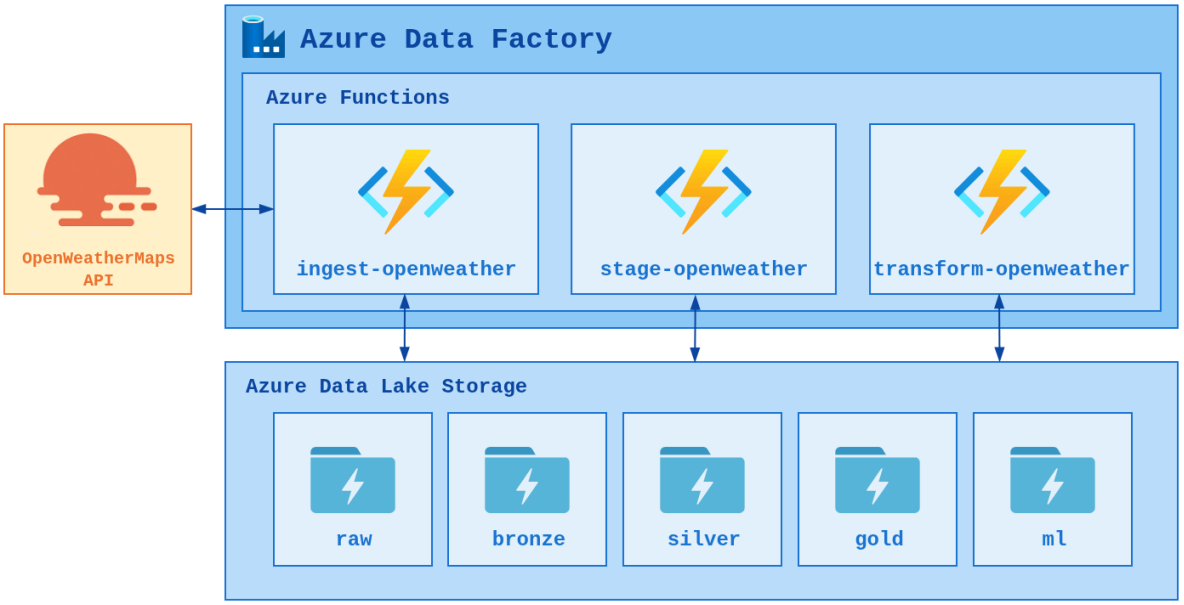


Figure 4. Summarizing diagram of the final Cloud Architecture.

3.1.2. Code Architecture

All the code generated for this project is public under the GPL-3.0 license[36] at github.com/EloiSanchez/openweather-reporting-pipeline/tree/master-thesis-delivery[37].

Hereinafter, references to particular directories or files of the source code in this repo will be shortened specifying only their relative path in respect to the root directory of this repository. Therefore, if directory `openweather-functions/src/destinations/` is mentioned, it should be understood as:

github.com/EloiSanchez/openweather-reporting-pipeline/tree/master-thesis-delivery/openweather-functions/src/destinations.

Even though the code was designed and implemented on a per-need basis, there were two requirements that were set at the beginning:

- It should be easily integratable with Azure functions, since it was decided that they would act as main units of processing power.
- Whatever functionality was needed, it should be created as a somewhat programmatic interface that would allow users to easily test it and modify the configuration of production executions. Although designing the code to fulfil this requirement adds development time

[36] "GNU General Public License Version 3."
[37] Sanchez Ambros, "EloiSanchez/Openweather-Reporting-Pipeline."

at the beginning of the process, scaling and maintenance becomes a much easier task in the long run.

With that in mind, three interfaces were created for the three main steps that would be run sequentially (Ingestion, Staging and Transformation), which are explained with more detail in sections 3.2 to 3.4.

Destination Interface

Besides the three interfaces mentioned above, a Destination Interface has been created to facilitate communication with a storage layer.

This interface provides two public implementations `LocalDirectory` and `ADLS`, both inheriting from the abstract class `BaseDestination`. Having both classes available and completely interchangeable makes it possible to very easily modify the production code to use a cloud ADLS instance or a local directory as storage, as long as the rest of the code was designed to work with the more general `BaseDestination`.

In order to authenticate with ADLS, credentials can be passed to the constructor of the `ADLS` class. However, if they are not passed, they will be looked for as environment variables.

Full source code for these implementations are available in the directory `openweather-functions/src/destinations/`, and their integration with the rest of the Interfaces will become clear in Sections 3.2 to 3.4.

Utility Module

Even though Python is a non-typed language by default, it also natively supports the use of type annotations and hints on variable and function definitions. Therefore, type annotations and hints have been used wherever deemed useful for clarity and distinctive type aliases have been created and organized in `openweather-functions/src/utils/types.py`.

Moreover, three custom classes (`DBModel`, `DictTable` and `Timestamp`) have been added in the same utils directory, since they are reused in multiple places in the codebase. Details on their implementations can be found in their respective files in directory `openweather-functions/src/utils/`.

3.1.3. Data Architecture

Given that the main objective of the project is to create a pipeline for data exploitation, this section is dedicated to providing a certain amount of detail on the data life cycle in the pipeline.

As has been specified above, and will be elaborated further in the following sections, there are three sequential processes in the pipeline that serve as natural checkpoints to evaluate how the data is structured at different stages.

Ingestion

Data is extracted from two endpoints, one providing a generalistic overview of the weather in a location, and the other providing details about the air quality. Both endpoints provide data at an hourly granularity and in a JSON format. Moreover, data can be requested for periods of up to one month at a time for a single location.

However, before loading the data into the database, it is restructured so that it is better organized using the hierarchical namespace functionality of ADLS. Therefore, a batching procedure, which can be seen implemented in the `batch_raw_data` method of the OpenWeather class (openweather-functions/src/ingest/openweather.py), produces batches partitioned by endpoint, day, and location, in this order, where each day includes the data of each location in a different file. In Figure 5, a schematic diagram of the data movement and transformation during the ingestion process is shown.

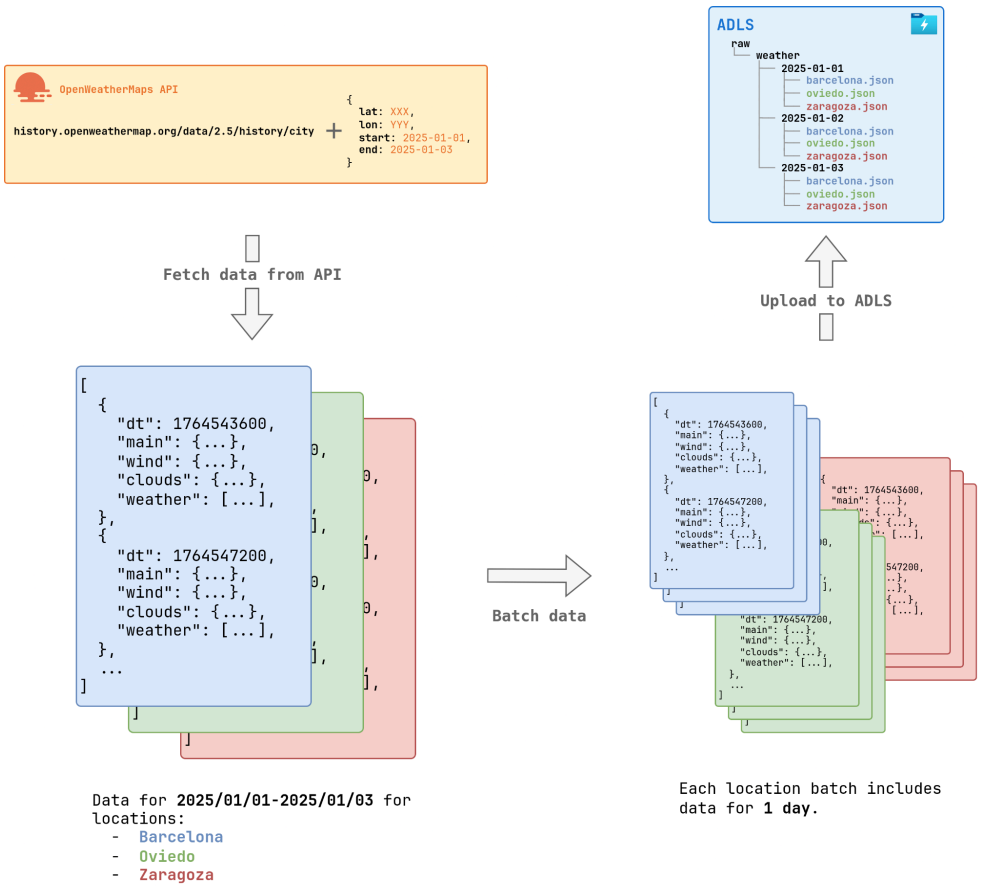


Figure 5. Schematic diagram showing the data flow for an ingestion process from three locations during the time period from 2025-01-01 to 2025-01-03, from one endpoint. In this example, data is loaded to an ADLS location, although other target locations could be used.

Staging

During the staging process, data is brought from the raw layer into the `bronze` layer in the ADLS. The `bronze` layer is the first that we logically consider to be part of the analytical database and, therefore, our objective in this step is to go from a semi-structured dataset into a columnar, analytics-oriented dataset that can be later used for further transformations.

The parquet file format is a compressed, column-oriented, open source file format designed for efficient data storage and retrieval. It was built from scratch by the Apache Software Foundation with the aim of designing a file format that was both storage efficient and provided efficient reads for big-data analytics workloads[38].

The flattening of the data from a semi-structured JSON file is relatively complex, due to the arbitrarily nested structure of JSON. Moreover, this is further complicated by the fact that JSON values can, in turn, contain arrays of data.

In order to facilitate understanding this process an example using simplified mock data can be seen in Figure 6. The staging process will read through the JSON files to load into the `bronze` layer and infer the number of columns that have to be created in the corresponding tabular structure that has to be created. Separators used for nested column names are double underscores, to clearly differentiate them from the common single underscore that may be used in the source data. Moreover, if a value from the current row includes an array of values, a secondary table, linked to the first one via the original row id, has to be created.

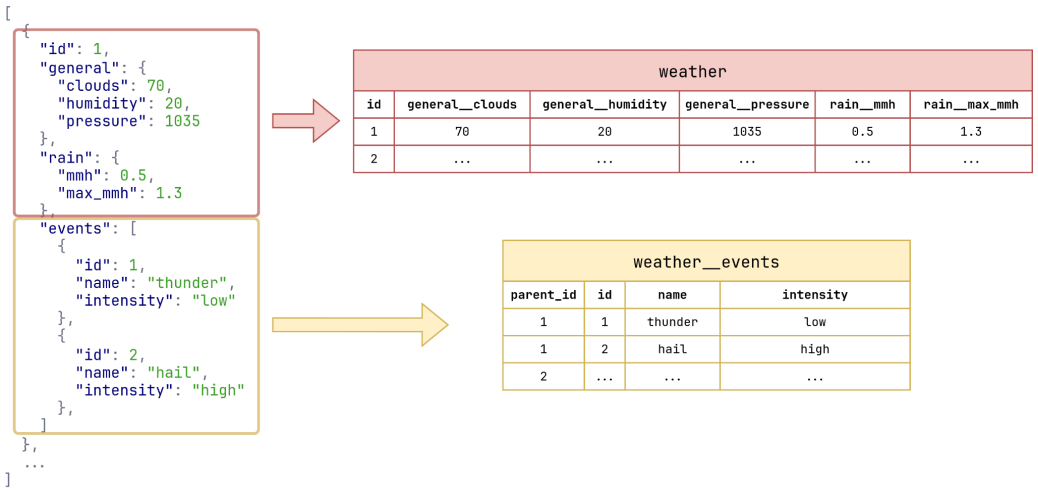


Figure 6. Flattening example of weather JSON data being flattened into a base table called `weather`. The exemplified JSON shows only one row of data with an events key that contains a nested array that needs to be loaded into a secondary linked table.

[38] Apache Software Foundation, “The Parquet File Format.”

The resulting tables from the flattening process will be saved into a target directory (`bronze`, in the case of this project) as parquet files.

For implementation details refer to Section 3.3.

Transformation

Once data is loaded into the "analytical" section of the database, all models are of a common tabular form, and all further datasets are also stored in parquet files.

In Figure 7, the data flow is shown through the different directories, or layers, of the ADLS production container. When advancing from `bronze` into `silver`, and later into `gold`, data becomes more clean and aggregated, and different KPIs are calculated and stored in `gold`.

Table `daily_general_report`, stored in the `gold` layer, includes a daily aggregation of all weather and air pollution metrics for each of the locations. Furthermore, a directory is especially created for Machine Learning ready datasets. Extensive details of the obtained models in the `gold` and `ml` directories are provided in section 3.6.

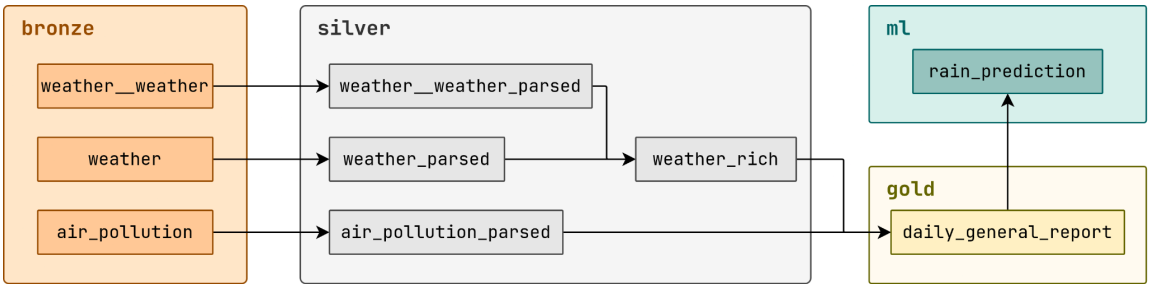


Figure 7. Schematic diagram showing the data models created during the Transformation step of the pipeline and the dependencies among them.

3.2. Ingestion

The Ingestion process is the first one to be executed in the pipeline and it has two main objectives. First, it needs to request the required data from the OpenWeather API[39]. Then, it needs to save the data in the specified Destination provided during configuration.

Figure 8 shows an overview of the ingestion process, as well as the design of the Ingestion Interface and its relation to the OpenWeather class. Detailed description follows in this section.

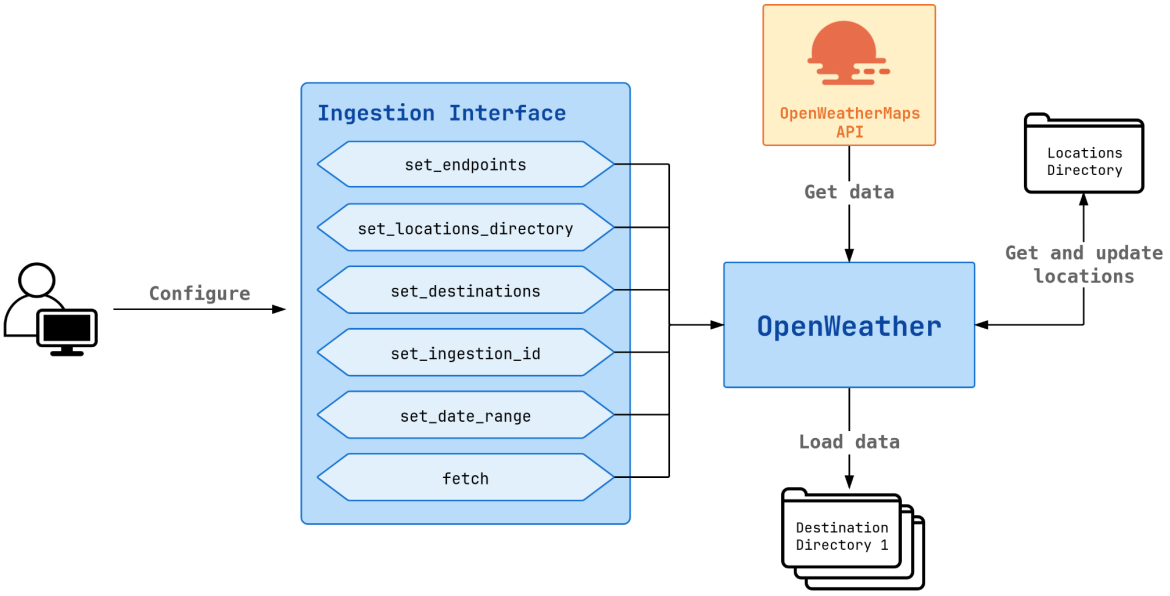


Figure 8. High level diagram of the `OpenWeather` class implementation and the provided user-facing Ingestion interface.

As with the rest of the main processes that take place in the pipeline, the entrypoint of the execution is a function defined in the `openweather-functions/function_app.py` script. In this case, the function that handles ingestion, and the one that will be called by Azure Functions when performing the ingestion step, is called `ingest_openweather`, and is properly decorated as specified in the Azure Functions documentation[40].

It is required that the `OPENWEATHER_SECRET_KEY` is available as an environment variable to authenticate with the OpenWeatherMap API. Alternatively, the key can be passed to the `OpenWeather` class constructor during instantiation.

[39] "OpenWeatherMap."
[40] "Python Developer Reference for Azure Functions."

In addition, in order to fetch the OpenWeatherMap API, it is required to provide information about the locations to fetch the data from. This is configured in a JSON file that includes a list of key-pairs with at least the following parameters:

- `search_name`: The name that will be used to search for the specific latitude and longitude required by the OpenWeatherMap API.
- `country_code`: The country where the `search_name` should be looked for.

With this information, requests to the direct Geocoding API[41] to find the actual `name` of the location as well as its latitude (`lat`) and longitude (`lon`) will automatically be made. If these three last parameters are already defined for a specific location in the configuration file, then the fetch of the Geocoding API will be skipped. Moreover, the file will be rewritten with the extra information from the Geocoding API in order to skip unnecessary requests in subsequent runs that reuse this file.

The Ingestion interface is the only one that includes a Command Line Interface (CLI) to interact with it. This CLI was created to work as a standalone for ingesting data from the OpenWeatherMap API and details of usage, including examples with different types of Destinations, are available in the repository `README.md`. The full implementation of the ingestion CLI can be seen in `openweather-functions/src/ingest/cli.py`.

The CLI is, in fact, a wrapper for the `OpenWeather` class which can be used as an imperative Interface via Python. In Snippet 1, an extract showing how the imperative `OpenWeather` class is used in the production pipeline at the moment of writing. Further details and context can be seen in `openweather-functions/function_app.py`.

A somewhat extensive explanation of the snippet is provided now, which will be skipped on the explanations for Staging and Ingestion, given that the main characteristics are common for all three interfaces.

```
OpenWeather()
.set_location_directory(ADLS(directory="locations"))
.set_destinations([ADLS(directory="raw")])
.set_endpoints("all")
.set_date_range(start_date=None, end_date=None)
.set_ingestion_id(req.headers.get("run_id"))
.fetch()
```

Snippet 1. Extract of the high-level configuration and execution of the Ingestion API via the `OpenWeather` class.

As can be seen, the `OpenWeather` class, the implementation of which can be seen in `openweather-functions/src/ingestion/openweather.py`, provides a clear and concise way of working, which makes performing changes to the ingestion process of the pipeline a

[41] "Geocoding API - OpenWeatherMap."

relatively easy task. In general, there is a setter method available for each configuration setting. Each of the setter methods triggers the `OpenWeather` instance to perform all the back-end modifications necessary so that, on the moment the `fetch()` method is called, the ingestion can take place.

For instance, `set_endpoints("all")` indicates that the two available endpoints that are available should be fetched, general weather[42] and air pollution[43]. As another example, the `set_date_range(...)` method accepts passing `None` as parameters for the dates. If `start_date` is not passed, then the `OpenWeather` instance will check the last date already available in the Destination and fetch new data from that point onwards, limiting the amount of requests performed to the strictly necessary. If `end_date` is not passed, instead, data will be fetched until the day before the ingestion is performed.

Furthermore, Snippet 1 shows how Destinations are designed to be used. Two `ADLS` instances are used in this case, one (`ADLS(directory="locations")`) to indicate where the location configuration file should be looked for, and the other one (`ADLS(directory="raw")`) to indicate where the fetched data should be loaded into. In order to change either of these to work locally, the class constructor could simply be changed from `ADLS` to `LocalDirectory`. Since both are implementations of the abstract `BaseDestination` class, and the `OpenWeather` class has been designed to be compatible with any implementation of `BaseDestination`, the ingestion process will continue to work completely locally.

In addition, this allows the `OpenWeather` class, and the rest of the classes used during Staging and Transforming to be agnostic to the specific instance of Destination, which is important when considering scalability. Therefore, to give an example, in order to support S3 buckets by AWS, we could simply implement an `S3Bucket` class compatible with `BaseDestination`, and the pipeline would continue working just by changing the `ADLS` instances to `S3Bucket` instances.

[42] “Historical Weather API - OpenWeatherMap.”
[43] “Air Pollution API - OpenWeatherMap.”

3.3. Staging

The staging process takes the data from the raw directory, where it is stored as semi-structured JSON files, into the `bronze` directory of the analytical database. During this process, in addition, data is parsed and flattened into tabular data that is stored in the parquet file format. Figure 9 shows an overview of the Staging process, which is described in detail in the rest of this section.

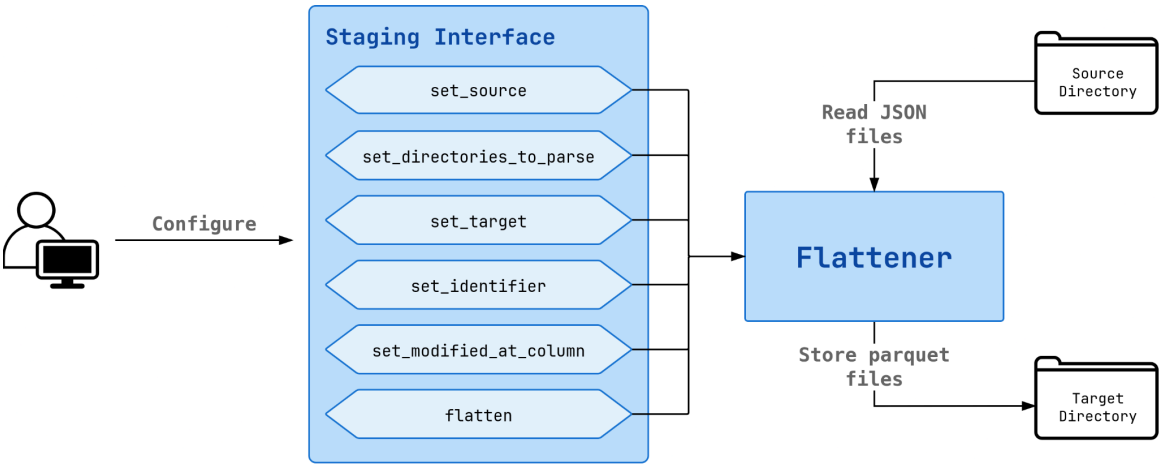


Figure 9. High level diagram of the `Flattener` class implementation and the provided user-facing Staging interface.

Therefore, in the project's architecture, once the Azure Ingest Openweather Function is completed, the Staging Openweather is called. This function will essentially construct the `Flattener` object, configure it via its imperative API and execute the flattening process, which will also load the data into the configured directory.

The production usage of the Staging Function can be seen in the entry-point for all functions of the project (openweather-functions/function_app.py) in the `stage_openweather` function definition. Similarly to the OpenWeather class used during ingestion, the Flattener class is designed to provide a lightweight and easy to understand syntax, which is shown in Snippet 2.

```
Flattener()
.set_source(ADLS(directory="raw"))
.set_directories_to_parse("weather", "air_pollution")
.set_target(ADLS(directory="bronze"))
.set_identifier(req.headers.get("run_id"), "staged_id")
.set_modified_at_column("staged_at")
.flatten()
```

Snippet 2. Extract of the high-level configuration and execution of the Staging API via the `Flattener` class.

In the same philosophy as with the `OpenWeather` class, the `Flattener` class is designed to be compatible with any Destination-like class, so we could change the `ADLS` destinations for `LocalDirectory` destinations in order to work locally, as well as with any other class that implements the `BaseDestination` abstract class.

Using the `set_identifier` and `set_modified_at_column` we can modify the tagging columns obtained in the flattened datasets, and via the `set_directories_to_parse`, we indicate which directories from the source destination to stage during the process.

The implementation of the `Flattener` is completely agnostic to the specific JSON schemas of the raw data, since it flattens the data dynamically inferring the different columns and their names.

In file `openweather-functions/src/transform/flattener.py`, details of the flattening process implementation are available.

Initially, when working with a limited amount of data, the DuckDB library was being used in order to serialize the data into the parquet file format[44]. However, at the beginning of December, when the code was brought to near its final state, a back-filling ingestion execution was performed to get data from the 1st of January of 2025 until the current date. It was then seen that the staging process was not scalable enough under bigger amounts of data, and multiple optimizations had to be made. One of the changes made was changing the DuckDB library to the polars library for creating the flattened relations and later loading them into parquet files.

[44] DuckDB, “An In-Process SQL OLAP Database Management System.”

3.4. Transformation

Finally, once the Staging Openweather function has finished and data has landed into the analytical section of the storage layer, the `bronze` directory, the Transform Openweather Azure Function is called.

This is the process where data gets transformed from raw to usable, populating the `silver`, `gold` and `ml` directories of the database. Following the same tactics as in the previous sections, an imperative API was designed in order to allow developers to easily add new models and transformations. Figure 10 shows a diagram of the Transformation process described in this section.

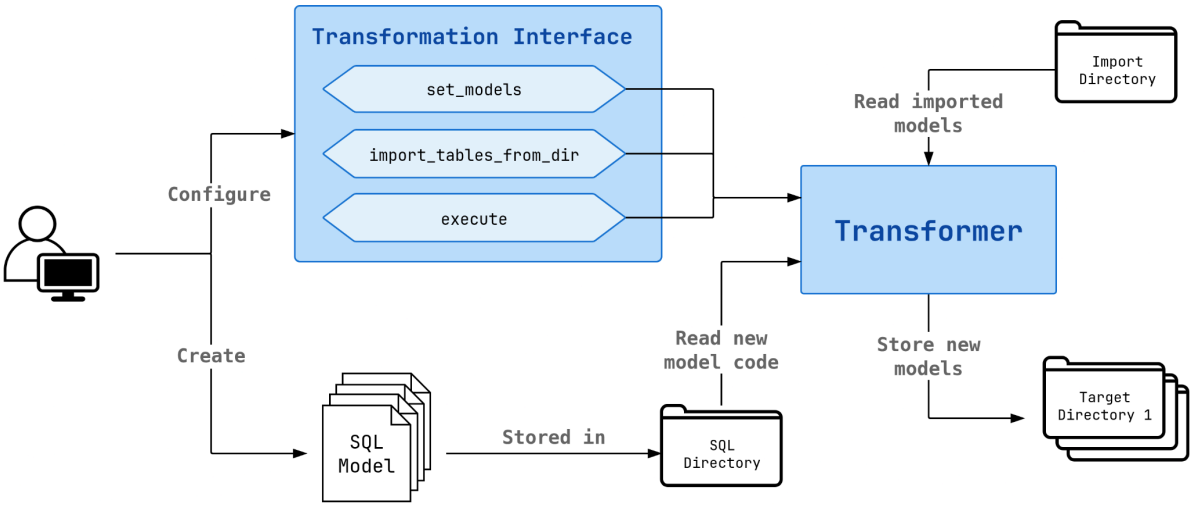


Figure 10. High level diagram of the `Transformer` class implementation and the provided user-facing Transformation interface.

Multiple libraries could be used as foundation for this final process, but since during the development process the DuckDB library was already being used during the Staging process, it was decided to also utilize it for transformations.

The DuckDB library provides a barebones interface to use with multiple programming languages, including Python. However, this interface is not as rich and complete as the ones provided by libraries like Pandas, Polars or Pyspark, among others. Therefore, it was decided that, since data was already in a flattened-tabular format in the database when it landed in the `bronze` directory, using SQL would be comfortable enough, as long as the Transformation API was designed with organization and scalability in mind.

As a result, the Transformer object was created so that a certain number of models could be set as pre-loaded and able to be used in later transformations. Transformations would be defined as plain SQL files, in a 1-level deep hierarchical directory structure, where their

parent directory would indicate which directory of the database would later host the generated table.

Snippet 3 shows an extract of how this class is currently being used in production in the functions entry point (openweather-functions/function_app.py), and will serve as an example to better understand the process.

```
con = duckdb.connect()

bronze = ADLS(directory="bronze")
silver = ADLS(directory="silver")
gold = ADLS(directory="gold")
ml = ADLS(directory="ml")

(
    Transformer(con)
    .set_models(
        [
            ("sql/silver/weather_parsed.sql", silver),
            ("sql/silver/weather__weather_parsed.sql", silver),
            ("sql/silver/air_pollution_parsed.sql", silver),
            ("sql/silver/weather_rich.sql", silver),
            ("sql/gold/daily_general_report.sql", gold),
            ("sql/ml/rain_prediction.sql", ml),
        ]
    )
    .import_tables_from_dir(bronze)
    .execute()
)
```

Snippet 3. Extract of the high-level configuration and execution of the Transformation API via the `Transformer` class.

Initially, a DuckDB connection is initialized. By default, this connection is created to a volatile in-memory database, but can be modified to become a permanent disk-written database if the requirement arises. This connection will be used by the `Transformer` to read and write models, allowing (non-cyclical) inter-dependency between models.

Next, the directories that are going to be used are initialized via the Destination interface, and, just like in the previously described Ingestion and Staging processes, we use `ADLS` directories in production but `LocalDirectory` or any other implementation of `BaseDestination` could be used interchangeably.

Via the `import_tables_from_dir` method we define which directory to pre-load tables from. In this case we use the `bronze` directory, populated with fresh data in the Staging process.

Then, a sequence of models to be created are passed in an orderly manner, indicating the path of the SQL files defining each model and the destination where each of these models is to be created. The SQL files can be seen in `openweather-functions/sql/`.

On the moment of execution, the Transformer will start executing the SQL queries passed and saving the results into new tables in the target directories, making them available for further SQL queries to be used, but also written for other purposes in a permanent manner.

Moreover, in `openweather-functions/src/transform/transformer.py` the source code implementation of the `Flattener` class is also available.

Using this imperative Transformer Interface, adding a new model, or performing changes to existing models is as easy as creating a new SQL file in the corresponding directory, or modifying an already existing one.

3.5. Orchestration

In the past sections the details of execution of the three Azure Functions (Ingestion, Staging and Transformation) have been shown. However, functions need to be automated in some way so that execution takes place in an orderly fashion at a schedule.

In order to do this, Azure Data Factory (ADF) has been utilized[45]. This tool is the main orchestration tool provided by the Azure Cloud suite, and provides an easy-to-use graphical interface to configure "pipelines" (Figure 11).

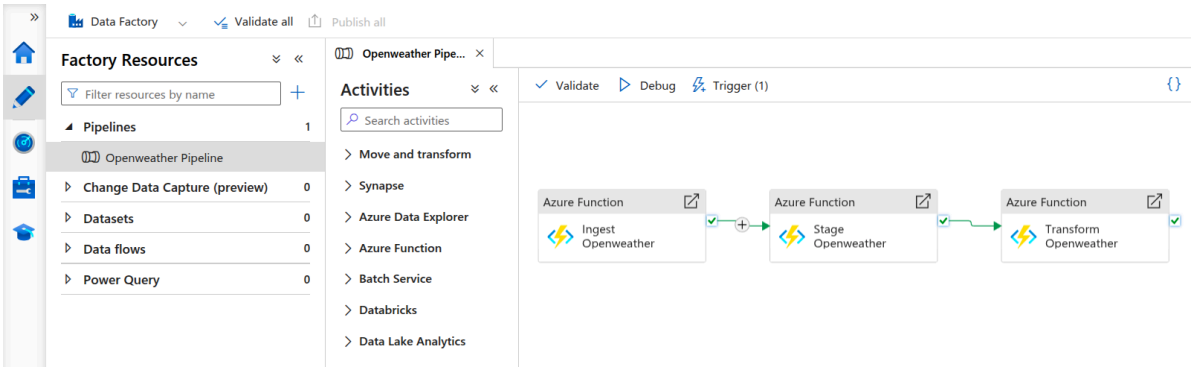


Figure 11. Openweather Pipeline seen in the user interface form Azure Data Factory.

Pipelines are aggregations of activities in the form of graphs or flow diagrams as can be seen in Figure 11. Each activity is a unit of execution that takes certain inputs, performs a certain task, and provides an output. ADF provides lots of different activities that integrate well with the rest of the microservices of Azure.

In our project, the Openweather Pipeline consists of three activities that perform calls to the Ingest, Stage and Transform Openweather functions in a sequential manner. After the execution of each function, ADF checks its output. If the execution is a success, the following function is then called for execution.

The input of each function is a dictionary of parameters, given that they are triggered via a standard GET request, that sends the pipeline run id. Each function then will handle this run id in the requests in a different manner, but is generally used to tag the rows of the dataset that have been processed by this execution. This allows linking specific batches of data in the obtained datasets to the original pipeline run that produced them, which is important for debugging, maintaining and auditing the overall pipeline.

Pipelines can then be assigned triggers that will start their execution. In the current case, a Scheduled Trigger is set, triggering the start of the pipeline at 08:30 (Amsterdam/Europe)

[45] "Azure Data Factory."

from Monday to Friday. The pipeline can then be monitored in the monitoring section of the ADF graphical user interface.

This marks the completion of all the main objectives that were proposed for the data pipeline, since, every weekday, ingestion, staging and transformation of new data is performed automatically. This ensures that, unless there are unexpected failures, clean and ready-to-use data lands in the `gold` and `ml` directories of the database on a daily basis.

3.6. Pipeline Exploitation

The previous sections detail how fresh data is made available on a daily basis in the different layers of the database (ADLS). Now, this data can be exploited for multiple purposes depending on the needs of the organization.

For instance, given that the pipeline provides daily weather data of different locations in Spain, we could create an application that would allow end users to graphically browse the historical weather and air pollution data. Besides hourly granularity of the data, statistical reports could be created via different aggregations and, even, correlation studies of data could be provided between the weather in a location and its air pollution metrics. End users for this application could be as diverse as the general public, if distributed via a phone or browser application, or governments and institutions via specialized dashboards of private or public access.

As proof of the usability of the data in the pipeline, and in order to provide an organic end of the completed project, a basic KPI table that could be used in daily Dashboarding or Reporting, and a Machine Learning study have been created.

3.6.1. KPI Table

During the Staging process (Section 3.3), data is introduced into the analytics section of the database in the `bronze` layer. The data is distributed in three tables, which originate from two endpoints:

- Historic weather endpoint
    - `weather` table: Includes hourly metrics for each location reporting general weather information such as precipitation, wind or pressure.
    - `weather__weather` table: For each hour, the weather endpoint provides an array of further qualitative detail of the weather at that hour. Multiple entries could theoretically be included in this array, with key `weather`, and during the flattening process, they are split into this secondary table. However, in practice, this is seldom used by the API.
- Air pollution endpoint
    - `air_pollution` table: Includes hourly air pollution metrics for each location. The metrics include measurements of NH_3, NO, NO_2, O_3, PM_{10}, $PM_{2.5}$, SO_2 and overall air quality.

Moreover, all the tables include information added by the ingestion and staging processes:

- `ingested_at` and `staged_at` timestamps.
- `ingestion_id` and `staging_id`, indicating which ADF run generated each row.
- `file_path`, indicating the storing location of the path.
- `source`, for now simply indicating that the source is OpenWeatherMap. This column is added in case other sources would be added to the project.

These raw tables are parsed and cleaned in `silver` (refer to Figure 7) where the two weather tables are joined together to include both of their information on each row. These intermediate tables are created and stored in the `silver` layer because they are not meant to be accessed directly by analysts or reporters, but they are to be reused by other models in the transformation process if needed.

Then, in `gold`, a daily general report table is created by joining both weather and air quality information. Moreover, the data is aggregated on a daily granularity. Since new data comes in every weekday into the database with information of the previous days, this table grows by one row every day, except for Mondays where it grows by three rows (including data of the previous Friday, Saturday and Sunday).

The fact that the pipeline runs completely automated, on a schedule, and always getting the latest available data, ensures that this reporting table is always up to date. Further reports and/or dashboards that would be getting their data from this table would also have fresh and ready to use data.

The `daily_weather_report` table, therefore, shows averages of all the weather and air quality metrics found in the raw tables, as well as minimum and maximum values for some of them. An example of the table with limited data (from 2025-12-01 to 2025-12-07) can be seen in `data_example/gold/daily_general_report.parquet`.

3.6.2. Rain Prediction

Preparation of the Dataset

As mentioned above, besides the `daily_weather_report.parquet` dataset, a predictive Machine Learning model has been performed. The final pipeline provides data that can be used in many different ways, i.e., air quality metrics prediction given weather of past days, weather predictions, statistical analysis for trend and pattern discoveries…

A rain prediction model was chosen as the example to be done. The main objective of the model is to predict whether there is going to be any rain on the following day. In order to make the prediction, the model will only have access to the weather metrics of the current day.

First, the dataset had to be obtained. Since daily data was needed, the natural precursor of the final model was the previously obtained `daily_general_report`. The transformation performed is a simple projection that excludes all air quality related columns, and keeps only date and weather related information. The `lead` function is used in order to obtain information about precipitation on the following day. Special attention is paid to ensure this calculation is partitioned by locations. Moreover, a qualitative column is created from the precipitation on the following day, classifying the rain of the following day as:

- **dry**: If there is less than 0.5 mm of accumulated precipitation.
- **light**: If the maximum precipitation rate is less than 2.5 mm per hour.
- **moderate**: If the maximum precipitation rate is in the 2.5 mm to 7.5 mm per hour range.
- **heavy**: If the maximum precipitation rate is more than 7.5 mm per hour.

The specific transformation used for creating the `rain_prediciton.parquet` model is available in `openweather-functions/sql/ml/rain_prediction.sql`.

Machine Learning Notebook

With a model readily available, and always with fresh data, the Machine Learning study was performed, which is available in the `notebooks/` directory of the repository. Note that the focus of this project was not to obtain an accurate Machine Learning for rain prediction. Instead, this study should be taken as a demonstration that, indeed, the data provided by the pipeline that has been developed is of a high enough quality that it can be used for even Data Science projects such as this one.

Next, a description of the study and its results are presented.

Preparing Data

Initially, data is downloaded using the Destination interface described in Section 3.1.2. More specifically, given that the production dataset is stored in Azure, the `ADLS` class is used.

The report provided in this section was executed on the 23rd of December of 2025, and includes data from the 31st of December of 2024 until the 22nd of December of 2025 for the capitals of the 17 autonomous communities in the country of Spain. For more details refer to `ingestion_config/locations.json`.

A short exploratory analysis of the data is done, specifically meant to see how the different numerical variables are distributed (Figure 12). At the date of execution, there were 6172 rows in the dataset.

It can be seen that some variables follow close to normal distributions (pressure, degrees, temperature variables), while others present close to normal distributions with somewhat clear tails (humidity, wind variables). Moreover, we can see that the clouds variable closely

resembles a uniform distribution, if it was not for the peak at 0% coverage (also seen on wind gusts at 0 km/h). Moreover, we can also see artifacts on the min. and max. temperature, probably caused by how OpenWeatherMap reports these temperatures. It can also be seen that, except for a few days that are barely visible on the plots, the rain variables are closely accumulated at the low end of the distribution (0-2.5 mm per hour).

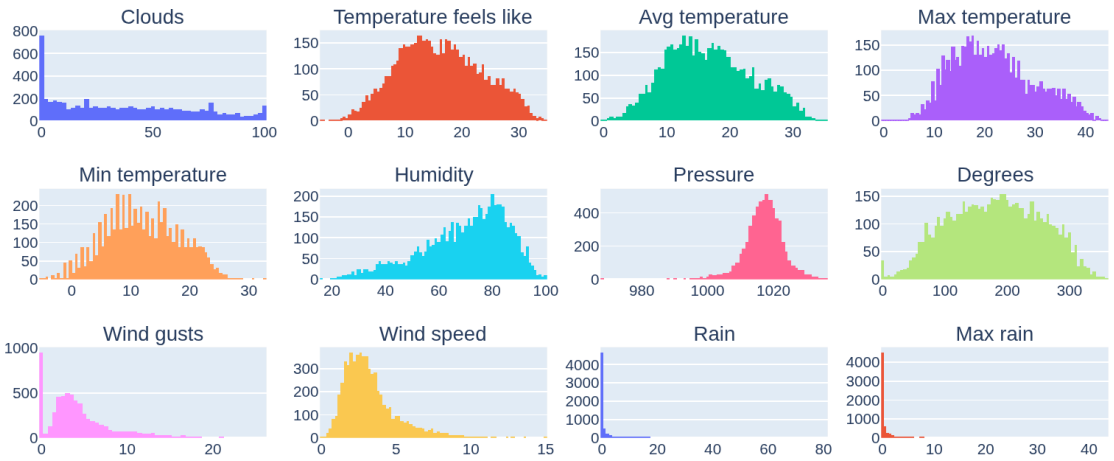


Figure 12. Numerical variable distributions of the rain prediction dataset.

Given the limited amount of data at the moment of execution of the study, location information is discarded, and we are left with 10 numerical columns that are going to act as predictor variables. These variables, in addition, are normalized via the common Z-score Normalization, which transforms them into distributions with 0 mean and a variance of 1.

Moreover, a new column is created that simply contains the values “dry” or “rain”, according to the qualitative rain variable in the dataset (see previous section). This limits the amount of possible values that have to be predicted by the trained models to two.

The dataset is then split, 80% for training (4937 rows) and 20% for testing (1235 rows). Given that the dataset is largely unbalanced in favor of “dry” days over “rain” days, the Adaptative Synthetic (ADASYN) over-sampling algorithm is performed on the training data set, which increases its size to 7571 rows[46].

Model Training and Evaluation

Two classifying models are selected to be trained and evaluated, Random Forest[47] and Gradient Boosting[48]. Both models are common ensemble models that train multiple simple

[46] He et al., “ADASYN.”
[47] Geurts et al., “Extremely Randomized Trees.”
[48] Friedman, “Greedy Function Approximation.”

models (100 in this case) that all make predictions during inference. The predictions of the simple models are then aggregated to obtain a final prediction.

The implementation of the ADASYN algorithm is provided by the imbalance-learn Python library[49]. The Random Forest and the Gradient Boosting Implementations are provided by the Sci-Kit Python library[50].

The confusion matrices for both models are presented in Table 1. It is quite apparent that, even though ADASYN was used in order to counteract the inherent imbalancedness of the dataset, it was not enough, and both models tend to predict the “dry” label when it should not be.

		Predicted Dry	Predicted Rain
Random Forest	True Dry	883	44
	True Rain	246	62
Gradient Boosting	True Dry	880	47
	True Rain	264	44

Table 1. Confusion matrices for trained models Random Forest and Gradient Boosting.

More detailed metrics are presented in Table 2. Even though both models show quite similar behaviour, obtaining results that are pretty similar in all calculated metrics, Random Forest seems to present a slightly better behaviour when predicting rain (62 correct rain predictions compared to 44 from Gradient Boosting). This produces a slight improvement on the Sensitivity of the model, from around 14% for Gradient Boosting to around 20% for Random Forest.

Further improvements are definitely possible with modifications on the training procedure. Hyper parameter tuning and other ways to account for the imbalancedness of the dataset can be introduced to improve the models behaviour. However, the focal point of the project was to design and implement an Automated Cloud Data Pipeline and show that the resulting data could be used for further applications such as KPI reporting and Machine Learning studies.

A more detailed discussion on potential improvements and next steps is presented in Section 4.2.

[49] Lemaître et al., “Imbalanced-Learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning.”
[50] Pedregosa et al., “Scikit-Learn: Machine Learning in Python.”

	Random Forest	Gradient Boosting
Accuracy	76.5 %	74.8 %
Balanced Accuracy	57.7 %	54.6 %
Precision (PPV)	58.5 %	48.4 %
Sensitivity (TPR)	20.1 %	14.2 %
Specificity (TNR)	95.3 %	94.9 %

Table 2. Evaluation metrics for trained models Random Forest and Gradient Boosting.

3.7. Financial Analysis

As mentioned in the previous sections, the pay-per-use model of cloud infrastructure is one of its big pillars. One of the big selling points is the fact that organizations do not have to pay infrastructure upfront, and all maintenance is left to the providers. Therefore, in this section we will analyze the costs of both developing and running the pipeline during these past months.

First of all, let it be emphasized that Azure was picked to be the cloud provider for this project due to its Student plan, which provides a 100$ USD of usage for free during one year. Moreover, some of the microservices have free usage available by default, as is the case of Azure Functions.

That being said, a distinction has been made between the development phase and the running phase. During the biggest part of this project (which started at the beginning of September) there has been a somewhat active development of the infrastructure, including building proof of concepts and tests of different kinds. The costs associated during this period are referred to as Development Cost, and are typically difficult to predict, since it depends on many factors that can vary strongly. Besides the development phase, there is an Operational or Running Cost. This is the cost of maintaining the infrastructure running without including new features or producing big changes in volumes of data. Besides, development and running costs may take place simultaneously, if production is released while development takes place at the same time.

In order to facilitate the analysis, the pipeline was left to run with minor human operations after the 14th of December, except for the 17th of December where some data was downloaded from ADLS. This allows us to infer the running cost of the pipeline.

As can be seen in the top left of Figure 13, the cost of resources since the creation of the Azure subscription on the 1st first of January until the end of the Development Phase was 4.18€ Moreover, it can be seen that most of its cost comes from the test performed at the beginning of October setting up a Databricks account for the transformation of the data. It was due to this reason that Databricks was shut down and alternatives were looked for, since a simple extrapolation would show that, even without running any transformation in Databricks, the cost of the full project at the end of the year would be of almost 100€.

In the top right of Figure 13, the costs for the same period of time but excluding the costs associated with Databricks are shown, amounting in this case to 0.58€. In addition, out of this cost, most of it comes from the ADLS usage of two days in the last week of the Development Phase. These two days correspond to two attempts of a full ingestion of data from the 1st of January to the day when the tests were run and the full list of 17 locations. Previous to that, the pipeline was running with a few days of data and only a few locations, which was enough for development purposes.

Figure 13. Daily costs of the Azure Cloud subscription grouped by Resource types. Title shows detail of data displayed and the total cost of the period selected is shown in between parentheses. Top left and top right: Ranges from 1st October to 14th December 2025. Bottom left: Ranges from 1st October to 22nd December 2025. Bottom right: Ranges from 15th to 22nd December 2025.

In the bottom right of Figure 13, we can see the range of days after all the data was ingested and the 17 locations were being fetched every new day. Even though at the time of writing this, there has only been a full week of operational execution and with a small deviation on the 17th of December due to manual downloads of data from ADLS, it can be seen that the cost of running the pipeline and getting new data is between 0.05 and 0.10 € per week. It is also visible that on the weekends (20th and 21st of December in the Figure) the costs are essentially nothing, given that the pipeline only runs during weekdays.

Finally, on the bottom left of Figure 13, the total cost of the project can be seen, since the Cloud account was started up until the 22nd of December, amounting to 4.26 €. This is safely in the acceptable range that was set out initially of less than 100$ USD in total. Moreover, taking into account this budget, the pipeline could be expected to run for months, and even years, without running out of budget. Of course, some considerations would have to be taken into account to make an accurate prediction, since the volume of data stored in ADLS will slowly but continuously increase and market dynamics can affect both the pricing of the used microservices and the exchange rate of the Euro and the United States Dollar, among others.

Chapter 4

Conclusions and Next Steps

4.1. Conclusions

An end-to-end automated data pipeline has been designed and implemented. In addition, the codebase has been designed to run the pipeline both locally and in the cloud. More specifically, the presented production pipeline has been hosted using the Azure Cloud provider.

An ingestion CLI tool, as well as an imperative interface to facilitate user interaction, has been created to ingest data from the OpenWeatherMap API in a flexible and configurable manner. A Staging and a Transformation interface have also been developed, which take the raw data from ingestion and transform it into clean and usable data.

Using the aforementioned interfaces, the raw data from OpenWeatherMap has been used in order to obtain a table including multiple KPIs, or statistical metrics, regarding the weather and air pollution of 17 different locations in Spain since the beginning of 2025 until the current date.

A Machine Learning dataset has been manufactured with the aim of training models for rain prediction, although the data stored in the database could be further used to manufacture datasets for several other purposes. Two models, Random Forest and Gradient Boosting, have been trained in a showcasing Machine Learning study for rain prediction, further proving that, indeed, the main objective of the data pipeline of fetching, cleaning and transforming raw data into usable data has been achieved.

Finally, the data pipeline has been automated to obtain new data every workday, providing fresh data for all subsequent applications without requiring any human interaction. The total cost of both development and orchestrated execution have been analysed and shown to be well within the budget that was available.

4.2. Next Steps

Even though all major aims of the project have been successfully achieved, there are lots of new features and improvements that could be implemented in all parts of the pipeline to increase its efficiency and robustness and, generally, provide even more and more diverse data. Next, a not all-inclusive list of additional features that could be implemented is provided roughly divided by pipeline sections.

4.2.1. Ingestion

At present, the Ingestion interface allows two endpoints to obtain the raw data for the project, but the OpenWeatherMap API provides dozens of different endpoints with various data which could be supported as well.

The CLI could be re-designed in a top-down approach to make it even more configurable and scalable in case other cloud providers are supported. Improved documentation for it could be provided, and it could even be packaged and distributed as a standalone application or Python library.

4.2.2. Staging

Some changes had to be made when going from a small amount of data during development to a significantly larger amount when starting to run the pipeline in production. It is yet to be researched whether this particular process of the pipeline could withstand data volumes of higher orders of magnitude.

Effort could be invested into removing the current Polars Python library dependency and using the standard Python library instead for parquet serialization.

4.2.3. Transformation

Given that Databricks was proven to be too expensive for the current budget, an alternative to be able to transform data in a somewhat modular and comfortable manner for the user was built from scratch with the Transforming Interface. There are hundreds of different tools available for this, from open source libraries like dbt, to paid ones like Coalesce. The effort required to improve the current Transformation interface into something that could compete with the other tools in the market is expected to be really high, and it could be more worthwhile to completely migrate to an already built and maintained tool.

4.2.4. Orchestration

Automated runs of the pipeline at a schedule have been implemented with ADF, which allows monitoring the executions of the pipeline via its browser web application. However, this requires the user to log into the web application to see the result of the pipeline.

The natural step forward on this part of the pipeline is to set up automatic notifications that would warn on pipeline execution failures and/or successes. Notifications could be set to be received in multiple destinations (phone, email, messaging applications…). Furthermore, multiple users could be set to receive the notifications, from technical owners that should fix the pipeline to end-users that could be warned if the data shown in their applications is stale.

Chapter 5

Glossary

3NF – Third Normalized Form

ADASYN – Adaptive Synthetic Sampling Approach

ADF – Azure Data Fabric

ADLS – Azure Data Lake Storage

AI – Artificial Intelligence

API – Application Programming Interface

AWS – Amazon Web Services

CLI – Command Line Interface

GCP – Google Cloud Platform

GUI – Graphical User Interface

IoT – Internet of Things

IT – Information Technology

KPI – Key Performance Indicator

ML – Machine Learning

PPV – Positive Predictive Value

SaaS – Software as a Service

TPR – True Positive Rate

TNR – True Negative Rate

Chapter 6

References

"Air Pollution API - OpenWeatherMap." Accessed October 9, 2025. https://openweathermap.org/api/air-pollution.

Apache Software Foundation. "The Parquet File Format." Accessed December 21, 2025. https://parquet.apache.org/docs/file-format/.

A/RES/70/1 Transforming Our World: The 2030 Agenda for Sustainable Development. September 25, 2015.

Arthur, Charles. "Tech Giants May Be Huge, but Nothing Matches Big Data." Technology. *The Guardian*, August 23, 2013. https://www.theguardian.com/technology/2013/aug/23/tech-giants-data.

"Azure Data Factory." Accessed December 22, 2025. https://learn.microsoft.com/en-us/azure/data-factory/.

"Azure Data Lake Storage Introduction - Azure Storage." Accessed December 15, 2025. https://learn.microsoft.com/en-us/azure/storage/blobs/data-lake-storage-introduction.

"Azure for Students | Microsoft Azure." Accessed December 15, 2025. https://azure.microsoft.com/en-us/free/students.

Bashir, Noman, Priya Donti, James Cuff, et al. "The Climate and Sustainability Implications of Generative AI." *An MIT Exploration of Generative AI*, MIT, March 27, 2024. https://mit-genai.pubpub.org/pub/8ulgrckc/release/2.

Databricks. "What Is a Medallion Architecture?" March 9, 2022. https://www.databricks.com/glossary/medallion-architecture.

"DATOS Y BARCELONA | Ciencia e Innovación | Ajuntament de Barcelona." Accessed October 9, 2025. https://www.barcelona.cat/barcelonaciencia/es/datos-y-barcelona.

Deloitte Insights. "Why Organizations Are Moving to the Cloud." Accessed October 10, 2025. https://www.deloitte.com/us/en/insights/industry/technology/why-organizations-are-moving-to-the-cloud.html.

DuckDB. "An In-Process SQL OLAP Database Management System." Accessed December 21, 2025. https://duckdb.org/.

Font, Juan. "A Complete Newbies Primer on the Data Center Ecosystem." Forbes. Accessed October 29, 2025.

https://www.forbes.com/councils/forbestechcouncil/2025/01/14/a-complete-newbies-primer-on-the-data-center-ecosystem/.

Friedman, Jerome H. “Greedy Function Approximation: A Gradient Boosting Machine.” *The Annals of Statistics* 29, no. 5 (2001): 1189–232. https://doi.org/10.1214/aos/1013203451.

“Geocoding API - OpenWeatherMap.” Accessed December 17, 2025. https://openweathermap.org/api/geocoding-api.

Geurts, Pierre, Damien Ernst, and Louis Wehenkel. “Extremely Randomized Trees.” *Machine Learning* 63, no. 1 (2006): 3–42. https://doi.org/10.1007/s10994-006-6226-1.

“GNU General Public License Version 3.” *Open Source Initiative*, n.d. Accessed December 17, 2025. https://opensource.org/license/gpl-3-0/.

He, Haibo, Yang Bai, Edwardo A. Garcia, and Shutao Li. “ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning.” *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, June 2008, 1322–28. https://doi.org/10.1109/IJCNN.2008.4633969.

“Historical Weather API - OpenWeatherMap.” Accessed October 9, 2025. https://openweathermap.org/history.

Inmon, W H. *Building the Data Warehouse*. Wiley, 1990.

Kimball, Ralph, Laura Reeves, Margy Ross, and Warren Thornthwaite. *The Data Warehouse Lifecycle Toolkit*. Wiley, 1998.

Kosmyna, Nataliya, Eugene Hauptmann, Ye Tong Yuan, et al. “Your Brain on ChatGPT: Accumulation of Cognitive Debt When Using an AI Assistant for Essay Writing Task.” arXiv:2506.08872. Preprint, arXiv, June 10, 2025. https://doi.org/10.48550/arXiv.2506.08872.

Lemaître, Guillaume, Fernando Nogueira, and Christos K. Aridas. “Imbalanced-Learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning.” *Journal of Machine Learning Research* 18, no. 17 (2017): 1–5.

Linstedt, Dan. “Data Vault Series 1 – Data Vault Overview.” *TDAN.Com*, July 1, 2002. https://tdan.com/data-vault-series-1-data-vault-overview/5054.

“Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity.” *METR Blog*, July 10, 2025. https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/.

Monserrate, Steven Gonzalez. “The Cloud Is Material: On the Environmental Impacts of Computation and Data Storage.” *MIT Case Studies in Social and Ethical Responsibilities of Computing*, no. Winter 2022 (January 2022). https://doi.org/10.21428/2c646de5.031d4553.

Moore, Susan. “How To Create A Business Case For Data Quality Improvement.” Gartner, June 19, 2018.

https://www.gartner.com/smarterwithgartner/how-to-create-a-business-case-for-data-quality-improvement.

“OpenWeatherMap.” Accessed October 9, 2025. https://openweathermap.org/.

Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. “Scikit-Learn: Machine Learning in Python.” *Journal of Machine Learning Research* 12, no. 85 (2011): 2825–30.

Perspectives, Teradata. “Why Data Matters: The Purpose And Value Of Analytics-Led Decisions.” Forbes, October 15, 2020. https://www.forbes.com/sites/teradata/2020/10/15/why-data-matters/.

“Python Developer Reference for Azure Functions.” Accessed December 17, 2025. https://learn.microsoft.com/en-us/azure/azure-functions/functions-reference-python.

Reis, Joe, and Matt Housley. *Fundamentals of Data Engineering: Plan and Build Robust Data Systems*. First edition. O’Reilly, 2022.

Rodero, José A, José A Toval, and Mario G Piattini. “The Audit of the Data Warehouse Framework.” *Proceedings of the International Workshop on Design and Management of Data Warehouses* 19 (June 1999).

Salesforce. *73% of Business Leaders Believe Data Reduces Uncertainty and Drives Better Decisions — So Why Aren’t They Using It?* February 22, 2023. https://www.salesforce.com/news/stories/data-skills-research/.

Sanchez Ambros, Eloi. “EloiSanchez/Openweather-Reporting-Pipeline.” Accessed December 28, 2025. https://github.com/EloiSanchez/openweather-reporting-pipeline/tree/master-thesis-delivery.

Shafiq, Muhammad, Zhaoquan Gu, Omar Cheikhrouhou, Wajdi Alhakami, and Habib Hamam. “The Rise of ‘Internet of Things’: Review and Open Research Issues Related to Detection and Prevention of IoT-Based Security Attacks.” *Wireless Communications and Mobile Computing* 2022, no. 1 (2022): 8669348. https://doi.org/10.1155/2022/8669348.

Strubell, Emma, Ananya Ganesh, and Andrew McCallum. “Energy and Policy Considerations for Deep Learning in NLP.” arXiv:1906.02243. Preprint, arXiv, June 5, 2019. https://doi.org/10.48550/arXiv.1906.02243.

“TeamGantt.” Accessed December 25, 2025. https://www.teamgantt.com/.

US500. “S&P 500 Companies By Sector.” Accessed October 9, 2025. https://us500.com/sp500-companies-by-sector.

“Weather API - OpenWeatherMap.” Accessed October 29, 2025. https://openweathermap.org/api.

“What Is Elastic Computing - Definition | Microsoft Azure.” Accessed October 10, 2025. https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-elastic-computing.

“What Is Real-Time Data? | IBM.” July 29, 2025. https://www.ibm.com/think/topics/real-time-data.